

Relatório de Auditoria de Segurança

SuperHeroesApi

Data: 23 de setembro de 2026

Repositório: MarcosMuriloPJ/SuperHeroesApi

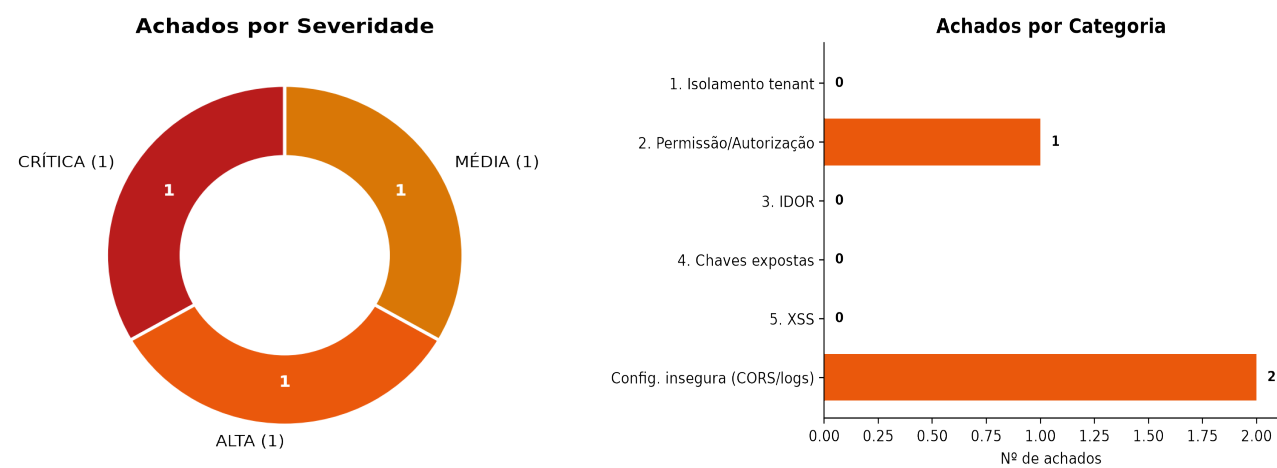
Escopo auditado: solução .NET 8 (ASP.NET Core Web API) composta por quatro projetos — SuperHeroesApi.Domain, SuperHeroesApi.Application, SuperHeroesApi.Infrastructure e SuperHeroesApi.WebAPI — implementando um CRUD de super-heróis e superpoderes sobre Entity Framework Core com provedor InMemory. Não há projeto de frontend, Dockerfile, pipeline de CI/CD, Helm chart ou Terraform neste repositório.

Nota metodológica — mapeamento das categorias para a stack detectada:

(1) Isolamento de tenant/dono: não há Supabase/RLS nem middleware de tenant nesta stack; buscou-se qualquer coluna de propriedade (owner_id/user_id/tenant_id) e filtro correspondente nas queries. Nenhum foi encontrado porque o domínio não modela usuários. (2) Permissão definida no navegador: como não há frontend no repositório, adaptou-se a verificação para o equivalente de backend — presença de [Authorize]/políticas de autorização do ASP.NET Core nos controllers. (3) IDOR: percorridos todos os handlers de HeroesController e SuperpowersController verificando o tratamento do parâmetro {id} frente a qualquer modelo de posse. (4) Chaves expostas: inspecionados appsettings*.json, código de configuração de infraestrutura (DbContext) e ausência de arquivos de deploy. (5) XSS: verificada a ausência de geração de HTML/e-mails no backend, dado que a API expõe somente JSON.

Resumo Executivo

Foram identificados **3 achados** classificados como: **1 crítico(s)**, **1 alto(s)**, **1 médio(s)** e **0 baixo(s)**. Quatro das cinco categorias solicitadas (isolamento de tenant, IDOR, chaves hardcoded e XSS) não se aplicam a este repositório pela natureza da stack (API sem conceito de usuário/tenant, sem frontend e sem arquivos de deploy) — ver seção 'Categorias não aplicáveis'. O achado central e mais grave é a **ausência completa de autenticação e autorização** em toda a API.



Pontos Fortes (verificados e corretos)

Item	Evidência
Swagger/OpenAPI restrito ao ambiente de desenvolvimento	Program.cs:66-73 — app.UseSwagger()/UseSwaggerUI() só é registrado dentro de if (app.Environment.IsDevelopment()), evitando exposição do contrato da API em produção.
HTTPS Redirection habilitado	Program.cs:77 — app.UseHttpsRedirection() força o uso de HTTPS.
Validação de modelo (Data Annotations) em todos os DTOs de escrita	CreateHeroDto.cs e UpdateHeroDto.cs aplicam [Required], [StringLength] e [Range], e os controllers verificam ModelState.IsValid (HeroesController.cs:85,118) antes de processar.
Validação de ID no path antes de qualquer consulta	HeroesController.cs:53,116,153 — GetHeroById, UpdateHero e DeleteHero rejeitam id <= 0 com BadRequest antes de tocar o repositório.
Regras de negócio centralizadas na entidade de domínio	Hero.cs — ValidateBusinessRules() garante datas e medidas consistentes antes da persistência, reduzindo risco de dados inconsistentes explorados posteriormente.
Nenhum segredo, senha ou chave de API hardcoded encontrado no código-fonte	Verificado em appsettings.json, appsettings.Development.json e todo o código de Infrastructure/WebAPI (busca por 'password', 'secret', 'apikey', 'token', 'connectionstring').
Nenhum ponto de renderização de HTML/template não sanitizado	A API expõe exclusivamente respostas JSON tipadas via DTOs — não há superfície de XSS server-side no código auditado.

Pontos Fracos (riscos centrais)

[CRÍTICA] Ausência total de autenticação/autorização nos endpoints de escrita — src/SuperHeroesApi.WebAPI/Controllers/HeroesController.cs:79,112,149

[ALTA] Política de CORS totalmente aberta definida como padrão, com nome de política divergente no middleware
— src/SuperHeroesApi.WebAPI/Program.cs:48-55,79

[MÉDIA] EnableSensitiveDataLogging() habilitado incondicionalmente (sem gate de ambiente) —
src/SuperHeroesApi.Infrastructure/Extensions/InfrastructureService.cs:19-20

Categorias Não Aplicáveis a Esta Stack

1. Banco sem tranca (RLS/tenant)

Não aplicável: a API não possui conceito de usuário, organização ou tenant. Heroes/Superpowers são um catálogo global único, sem coluna de propriedade (owner_id/tenant_id). Não há, portanto, mecanismo de isolamento a auditar; a ausência de autenticação em si já está coberta no achado F1.

3. IDOR

Não aplicável no sentido clássico: como não existe modelo de posse/usuário, não há 'objeto de outro usuário' a ser acessado indevidamente. O risco real equivalente (qualquer chamador acessa/altera/exclui qualquer registro por ID) já está coberto por F1, pois a causa raiz é a ausência total de autenticação, não uma falha de verificação de posse.

4. Chaves expostas (hardcode)

Nenhuma chave de API, senha, token JWT, segredo de webhook ou credencial hardcoded foi encontrada em código-fonte, appsettings.json, appsettings.Development.json ou InfrastructureService.cs. Não há arquivos Docker/CI/Helm/Terraform no repositório. O banco de dados é EF Core InMemory (UseInMemoryDatabase), sem connection string ou credenciais reais. Nenhum default inseguro de variável de ambiente (\${VAR:-default}) foi localizado.

5. Inputs sem tratamento (XSS)

Não aplicável: o repositório contém apenas uma API REST que retorna JSON puro via ActionResult/DTOs (serialização automática, sem geração de HTML). Não há projeto de frontend neste repositório, não há renderização de templates/e-mails, não há uso de innerHTML/dangerouslySetInnerHTML/v-html, eval ou new Function no código auditado.

Achados Detalhados

CRÍTICA**F1 — Ausência total de autenticação/autorização nos endpoints de escrita****Categoria:** 2. Permissão/Autorização (equivalente adaptado)**Arquivo:linha:** src/SuperHeroesApi.WebAPI/Controllers/HeroesController.cs:79,112,149

Os endpoints POST /api/heroes (linha 79), PUT /api/heroes/{id} (linha 112) e DELETE /api/heroes/{id} (linha 149) não possuem nenhum atributo [Authorize] nem qualquer outro controle de acesso. O projeto não registra nenhum esquema de autenticação (sem AddAuthentication/AddJwtBearer/AddIdentity em Program.cs ou InfrastructureService.cs). app.UseAuthorization() é chamado (Program.cs:81), mas não há política, esquema, papel (role) ou claim configurado para ser avaliado — logo, a chamada é um no-op de segurança. Qualquer cliente anônimo na rede pode criar, alterar ou excluir qualquer herói.

```
[HttpPost]
public async Task< CreateHero(...)
// sem [Authorize]
```

Condição de explorabilidade: Diretamente explorável hoje: basta enviar POST/PUT/DELETE para a API sem nenhum cabeçalho de autenticação. Não depende de feature flag.

ALTA**F2 — Política de CORS totalmente aberta definida como padrão, com nome de política divergente no middleware****Categoria:** Configuração insegura (CORS) — correlata à categoria 4**Arquivo:linha:** src/SuperHeroesApi.WebAPI/Program.cs:48-55,79

AddCors define uma AddDefaultPolicy com AllowAnyOrigin().AllowAnyMethod().AllowAnyHeader() (Program.cs:48-55) — extremamente permissiva. Em seguida, app.UseCors("VueAppPolicy") (linha 79) tenta aplicar uma política nomeada 'VueAppPolicy' que nunca foi registrada, deixando a política padrão permissiva sem efeito no momento. O risco é latente e de alta severidade: a correção óbvia que qualquer desenvolvedor fará (trocar para app.UseCors() sem nome, ou registrar 'VueAppPolicy' com a mesma regra AllowAny*) ativa imediatamente CORS totalmente aberto para todos os métodos (incluindo POST/PUT/DELETE) e todas as origens, permitindo que qualquer site controle a API a partir do navegador da vítima.

```
options.AddDefaultPolicy(policy => {
    policy.AllowAnyOrigin().AllowAnyMethod().AllowAnyHeader();
});
...
app.UseCors("VueAppPolicy"); // política inexistente
```

Condição de explorabilidade: Não ativa hoje devido ao nome divergente (bug funcional que hoje age como rede de segurança acidental), mas é uma condição insegura pronta para ser 'corrigida' de forma perigosa. Reportado como configuração insegura latente.

MÉDIA**F3 — EnableSensitiveDataLogging() habilitado incondicionalmente (sem gate de ambiente)****Categoria:** Configuração insegura / vazamento de dados em logs**Arquivo:linha:** src/SuperHeroesApi.Infrastructure/Extensions/InfrastructureService.cs:19-20

O DbContext é registrado com .UseInMemoryDatabase("SuperHeroesDB")..EnableSensitiveDataLogging() sem nenhuma checagem de ambiente (builder.Environment.IsDevelopment()). Essa opção instrui o EF Core a incluir valores de parâmetros de consulta (dados pessoais dos registros) nas mensagens de log e exceção. Hoje o provedor é InMemory (risco baixo), mas se o provedor for trocado para um banco real em produção sem remover essa linha, dados sensíveis podem vazar para logs de aplicação/observabilidade.

```
options.UseInMemoryDatabase("SuperHeroesDB")
.EnableSensitiveDataLogging();
```

Condição de explorabilidade: Requer troca futura do provedor InMemory por um banco real em produção mantendo essa configuração — risco condicional, mas fácil de ocorrer por descuido.

Recomendações Priorizadas

Prioridade	Recomendação
P1	Implementar autenticação (ex.: JWT Bearer) e autorização ([Authorize]/políticas de papel) em todos os endpoints de escrita (POST/PUT/DELETE) de HeroesController, e reavaliar se GET também deve exigir autenticação conforme o modelo de negócio.
P1	Corrigir a divergência de nome de política de CORS: registrar explicitamente a política 'VueAppPolicy' com WithOrigins().WithMethods(...).WithHeaders(...) em vez de AllowAny*, e remover a AddDefaultPolicy totalmente aberta.
P2	Remover ou condicionar EnableSensitiveDataLogging() a builder.Environment.IsDevelopment(), garantindo que nunca seja habilitado em produção.
P2	Adicionar testes automatizados de autorização (ex.: teste que POST/PUT/DELETE sem token retornam 401/403) para evitar regressão após a correção de P1.
P3	Ao evoluir para persistência real (SQL Server/Postgres), aplicar o princípio de isolamento de dados adequado ao modelo de negócio (ex.: se heróis passarem a pertencer a usuários/organizações, implementar filtro obrigatório por proprietário em todas as consultas e RLS no banco, se aplicável).
P3	Adicionar cabeçalhos de segurança HTTP padrão (HSTS, X-Content-Type-Options, Content-Security-Policy) via middleware, já que o pipeline atual não os define.

Issues para o GitHub

As issues abaixo estão prontas para copiar e colar no GitHub, uma por bloco delimitado.

--- ISSUE 1 ---

[Segurança] API sem autenticação/autorização permite criar, alterar e excluir heróis anonimamente

****Labels sugeridas:** security, critical**

Descrição do problema

Nenhum endpoint de escrita da HeroesController exige autenticação ou autorização. Não há esquema de autenticação registrado no pipeline (Program.cs) nem atributos [Authorize] nos controllers. app.UseAuthorization() está presente, mas sem nenhuma política configurada, não tem efeito prático.

Evidência

```csharp

src/SuperHeroesApi.WebAPI/Controllers/HeroesController.cs:79

[HttpPost]

public async Task<ActionResult<HeroDto>> CreateHero(...)

src/SuperHeroesApi.WebAPI/Controllers/HeroesController.cs:112

[HttpPut("{id}")]

public async Task<ActionResult<HeroDto>> UpdateHero(...)

src/SuperHeroesApi.WebAPI/Controllers/HeroesController.cs:149

[HttpDelete("{id}")]

public async Task<ActionResult> DeleteHero(...)

```

Impacto

Qualquer usuário anônimo pode adulterar ou destruir todo o catálogo de heróis (integridade e disponibilidade dos dados comprometidas). Em um cenário real com dados de negócio, isso representa comprometimento total de escrita da aplicação.

Sugestão de correção

- 1) Adicionar autenticação (ex.: JWT Bearer via AddAuthentication().AddJwtBearer()).
- 2) Adicionar [Authorize] (com papéis/políticas apropriadas) em CreateHero, UpdateHero e DeleteHero.
- 3) Reavaliar se os endpoints GET também devem exigir autenticação.

Critérios de aceite

- [] POST /api/heroes sem token retorna 401
- [] PUT /api/heroes/{id} sem token retorna 401
- [] DELETE /api/heroes/{id} sem token retorna 401
- [] Requisições com token válido e papel adequado continuam funcionando (teste de regressão)
- [] Testes automatizados cobrindo os casos acima adicionados ao projeto de testes

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Política de CORS aberta (AllowAny*) e nome de política divergente no middleware

Labels sugeridas: security, high

Descrição do problema

AddDefaultPolicy define CORS totalmente aberto (AllowAnyOrigin/AllowAnyMethod/AllowAnyHeader), mas app.UseCors() referencia uma política nomeada 'VueAppPolicy' que nunca foi registrada. Hoje isso faz com que nenhuma política seja aplicada, mas é uma armadilha: a correção mais provável (renomear ou remover o parâmetro) ativa CORS totalmente aberto para todos os métodos e origens.

Evidência

```
```csharp
src/SuperHeroesApi.WebAPI/Program.cs:48-55
options.AddDefaultPolicy(policy => {
policy.AllowAnyOrigin().AllowAnyMethod().AllowAnyHeader();
});
```

```
src/SuperHeroesApi.WebAPI/Program.cs:79
app.UseCors("VueAppPolicy");
```
```

Impacto

Se corrigido ingenuamente, qualquer site poderá fazer requisições cross-origin (incluindo POST/PUT/DELETE) contra a API a partir do navegador de qualquer usuário, ampliando a superfície de ataque, inclusive de CSRF via fetch/XHR.

Sugestão de correção

Registrar explicitamente uma política nomeada 'VueAppPolicy' com WithOrigins(<domínios confiáveis do frontend>), WithMethods(<métodos necessários>) e WithHeaders(<headers necessários>), removendo AllowAnyOrigin/AllowAnyMethod/AllowAnyHeader e a AddDefaultPolicy totalmente aberta.

Critérios de aceite

- [] Política CORS nomeada 'VueAppPolicy' registrada e referenciada corretamente
- [] Apenas origens explicitamente confiáveis são permitidas
- [] Requisição cross-origin de um domínio não listado é bloqueada (teste manual/automatizado)
- [] Frontend legítimo continua funcionando após a mudança

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] EnableSensitiveDataLogging() habilitado sem gate de ambiente

Labels sugeridas: security, medium

Descrição do problema

O registro do DbContext ativa EnableSensitiveDataLogging() incondicionalmente, sem verificar se o ambiente é Development. Essa opção inclui valores de parâmetros de consulta em logs e mensagens de exceção.

Evidência

```
```csharp
src/SuperHeroesApi.Infrastructure/Extensions/InfrastructureService.cs:19-20
options.UseInMemoryDatabase("SuperHeroesDB")
.EnableSensitiveDataLogging();
```
```

Impacto

Caso o provedor InMemory seja substituído por um banco real em produção sem remover essa configuração, dados sensíveis podem ser gravados em logs/telemetria, violando boas práticas de proteção de dados.

Sugestão de correção

Condicionar EnableSensitiveDataLogging() a builder.Environment.IsDevelopment(), ou removê-lo e usar logging de diagnóstico apenas localmente via configuração dedicada.

Critérios de aceite

- [] EnableSensitiveDataLogging() não é chamado quando ASPNETCORE_ENVIRONMENT=Production
- [] Build de produção não expõe valores de parâmetros SQL em logs de exceção

--- FIM ISSUE 3 ---